2.2.7. Пример анализа и тестирования требований

Поскольку наша задача состоит в том, чтобы сформировать понимание логики анализа и тестирования требований, мы будем рассматривать предельно краткий и простой их набор.



Отличный подробный пример требований можно найти в приложениях к книге Карла Вигерса «Разработка требований к программному обеспечению» («Software Requirements (3rd Edition) (Developer Best Practices)», Karl Wiegers, Joy Beatty).

Допустим, что у некоего клиента есть проблема: поступающие в огромном количестве его сотрудникам текстовые файлы приходят в разных кодировках, и сотрудники тратят много времени на перекодирование («ручной подбор кодировки»). Соответственно, он хотел бы иметь инструмент, позволяющий автоматически приводить кодировки всех текстовых файлов к некоей одной. Итак, на свет появляется проект с кодовым названием «Конвертер файлов».

Уровень бизнес-требований. Бизнес-требования (см. главу «Уровни и типы требований»[39]) изначально могут выглядеть так: «Необходим инструмент для автоматического приведения кодировок текстовых документов к одной».

Здесь мы можем задать множество вопросов. Для удобства приведём как сами вопросы, так и *предполагаемые ответы клиента*.



Задание 2.2.b: прежде чем читать приведённый ниже список вопросов, сформулируйте собственный.

- В каких форматах представлены текстовые документы (обычный текст, HTML, MD, что-то иное)? (*Понятия не имею, я в этом не разбираюсь.*)
- В каких кодировках приходят исходные документы? (*В разных.*)
- В какую кодировку нужно преобразовать документы? (*В самую удобную и универсальную.*)
- На каких языках написан текст в документах? (*Русский и английский.*)
- Откуда и как поступают текстовые документы (по почте, с сайтов, по сети, как-то иначе)? (*Это неважно. Поступают отовсюду, но мы их складываем в одну папку на диске, нам так удобно.*)
- Каков максимальный объём документа? (*Пара десятков страниц.*)
- Как часто появляются новые документы (например, сколько максимум документов может поступить за час)? (*200–300 в час.*)
- С помощью чего сотрудники просматривают документы? (*Notepad++.*)

Даже таких вопросов и ответов достаточно, чтобы переформулировать бизнес-требования следующим образом (обратите внимание, что многие вопросы были заданы на будущее и не привели к появлению в бизнес-требованиях лишней технической детализации).

Суть проекта: разработка инструмента, устраняющего проблему множественности кодировок в текстовых документах, расположенных в локальном дисковом хранилище.

Цели проекта:

- Исключение необходимости ручного подбора кодировок текстовых документов.
- Сокращение времени работы с текстовым документом на величину, необходимую для ручного подбора кодировки.

Метрики достижения целей:

- Полная автоматизация определения и преобразования кодировки текстового документа к заданной.
- Сокращение времени обработки текстового документа в среднем на 1–2 минуты на документ за счёт устранения необходимости ручного подбора кодировки.

Риски:

- Высокая техническая сложность безошибочного определения исходной кодировки текстового документа.

Почему мы решили, что среднее время на подбор кодировки составляет 1–2 минуты? Мы провели наблюдение. Также мы помним ответы заказчика на вопросы об исходных форматах документов, исходных и конечной кодировках (заказчик честно сказал, что не знает ответа), а потому мы попросили его предоставить нам доступ к хранилищу документов и выяснили:

- Исходные форматы: plain text, HTML, MD.
- Исходные кодировки: CP1251, UTF8, CP866, KOI8R.
- Целевая кодировка: UTF8.

На данном этапе мы вполне можем решить, что стоит заняться детализацией требований на более низких уровнях, т.к. появившиеся там вопросы позволят нам вернуться к бизнес-требованиям и улучшить их, если в этом возникнет необходимость.

Уровень пользовательских требований. Пришло время заняться уровнем пользовательских требований (см. главу «Уровни и типы требований»[{39}]). Проект у нас несколько специфичный — результатами работы программного средства будет пользоваться большое количество людей, но само программное средство при этом они использовать не будут (оно будет просто выполнять свою работу «само по себе» — запущенное на сервере с хранилищем документов). Потому под пользователем здесь мы будем понимать человека, настраивающего работу приложения на сервере.

Для начала мы создадим небольшую диаграмму вариантов использования, представленную на рисунке 2.2.g (да, иногда её создают **после** текстового описания требований, но иногда и **до** — нам сейчас удобнее сделать это сначала). В реальных проектах подобные схемы могут быть на несколько порядков более сложными и требующими подробной детализации каждого варианта использования. У нас же проект миниатюрный, потому схема получилась элементарной, и мы сразу переходим к описанию требований.

Внимание! Это — ПЛОХИЕ требования. И мы далее будем их улучшать.

Системные характеристики

- СХ-1: Приложение является консольным.
- СХ-2: Для работы приложение использует интерпретатор PHP.
- СХ-3: Приложение является кроссплатформенным.

Пользовательские требования

- Также см. диаграмму вариантов использования.
- ПТ-1: Запуск и остановка приложения.
    - ПТ-1.1: Запуск приложения производится из консоли командой «PHP converter.php параметры».

    - ПТ-1.2: Остановка приложения производится выполнением команды Ctrl+C.
- ПТ-2: Конфигурирование приложения.
    - ПТ-2.1: Конфигурирование приложения сводится к указанию путей в файловой системе.
    - ПТ-2.2: Целевой кодировкой является UTF8.
- ПТ-3: Просмотр журнала работы приложения.
    - ПТ-3.1: В процессе работы приложение должно выводить журнал своей работы в консоль и лог-файл.
    - ПТ-3.2: При первом запуске приложения лог-файл создаётся, а при последующих — дописывается.

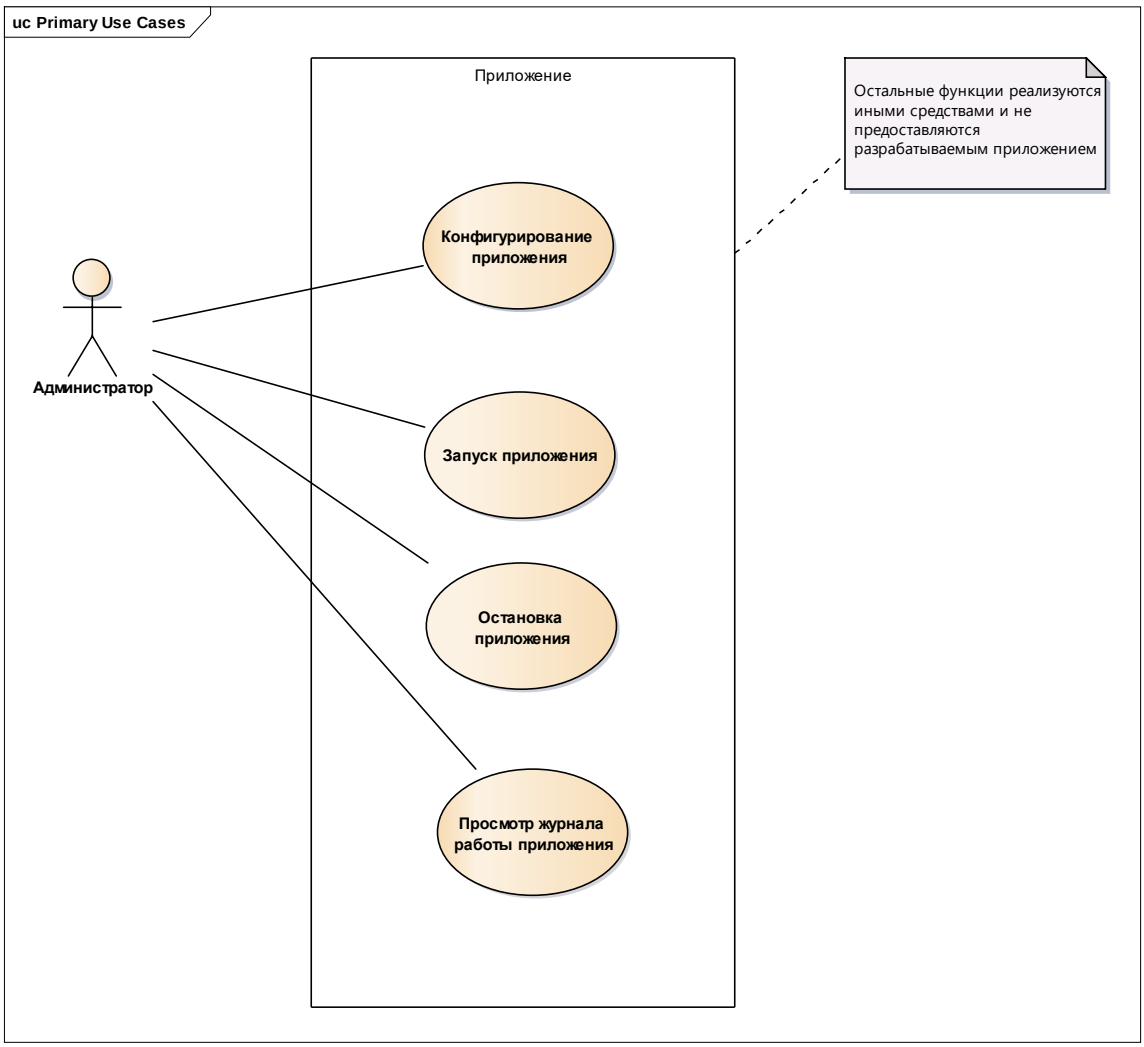


Рисунок 2.2.g — Диаграмма вариантов использования

Бизнес-правила

- БП-1: Источник и приёмник файлов
    - БП-1.1: Каталоги, являющиеся источником исходных и приёмником конечных файлов не должны совпадать.
    - БП-1.2: Каталог, являющийся приёмником конечных файлов, не может быть подкаталогом источника.

Атрибуты качества

- АК-1: Производительность
  - АК-1.1: Приложение должно обеспечивать скорость обработки данных 5 МБ/сек.
- АК-2: Устойчивость к входным данным
  - АК-2.1: Приложение должно обрабатывать входные файлы размером до 50 МБ включительно.
  - АК-2.2: Если входной файл не является текстовым, приложение должно произвести обработку.

Как будет сказано в главе «Типичные ошибки при анализе и тестировании требований»{63}, не стоит изменять исходный формат файла и форматирование документа, потому мы используем встроенные средства Word для отслеживания изменений и добавления комментариев. Примерный вид результата показан на рисунке 2.2.h.

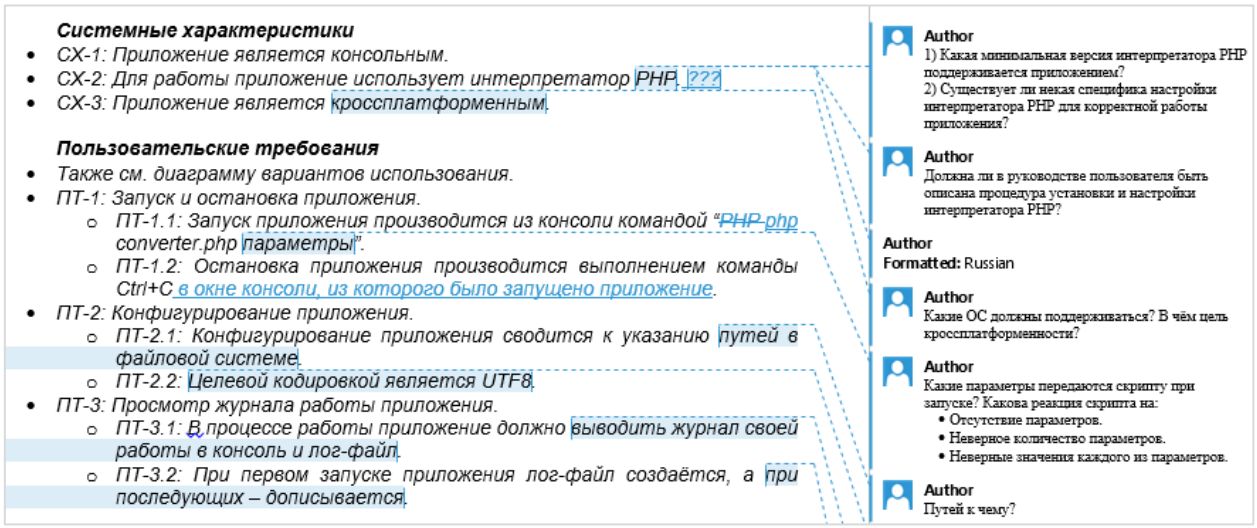


Рисунок 2.2.h — Использование средств Word для работы с требованиями

К сожалению, мы не можем в данном тексте применить эти средства (результат будет отображаться некорректно, т.к. вы сейчас, скорее всего, читаете этот текст не в виде DOCX-документа), а потому применим второй классический способ — будем вписывать свои вопросы и комментарии прямо внутрь текста требований.

Проблемные места требований отмечены подчёркиванием, наши вопросы отмечены *курсивом*, предполагаемые ответы заказчика (даже, если точнее, технического специалиста заказчика) — **жирным**. В процессе анализа текст требований примет вот такой вид.



Задание 2.2.c: проанализируйте предложенный набор требований с точки зрения свойств качественных требований{44}, сформулируйте свои вопросы заказчику, которые позволят улучшить этот набор требований.

Системные характеристики

- СХ-1: Приложение является консольным.
- СХ-2: Для работы приложение использует интерпретатор PHP.
  - *Какая минимальная версия интерпретатора PHP поддерживается приложением?* **(5.5.x)**
  - *Существует ли некая специфика настройки интерпретатора PHP для корректной работы приложения?* **(Наверное, должен работать mbstring.)**

  - *Настаиваете ли вы на реализации приложения именно на PHP? Если да, то почему.* **(Да, только PHP. У нас есть сотрудник, который его знает.)**
  - *Должна ли в руководстве пользователя быть описана процедура установки и настройки интерпретатора PHP?* **(Нет.)**
- СХ-3: Приложение является кроссплатформенным.
  - *Какие ОС должны поддерживаться?* **(Любая, где работает PHP.)**
  - *В чём вообще цель кроссплатформенности?* **(Мы ещё не знаем, на чём будет работать сервер.)**

Пользовательские требования

- Также см. диаграмму вариантов использования.
- ПТ-1: Запуск и остановка приложения.
  - ПТ-1.1: Запуск приложения производится из консоли командой «PHP *(возможно, здесь опечатка: должно быть php (в нижнем регистре))* **(Да, OK.)** converter.php параметры».
    - *Какие параметры передаются скрипту при запуске?* **(Каталог с исходными файлами, каталог с конечными файлами.)**
    - *Какова реакция скрипта на:*
      - *отсутствие параметров;* **(Пишет хелп.)**
      - *неверное количество параметров;* **(Пишет хелп и поясняет, что не так.)**
      - *неверные значения каждого из параметров.* **(Пишет хелп и поясняет, что не так.)**
  - ПТ-1.2: Остановка приложения производится выполнением команды Ctrl+C *(предлагаем дополнить это выражение фразой «в окне консоли, из которого было запущено приложение»)* **(Да, OK.)**.
- ПТ-2: Конфигурирование приложения.
  - ПТ-2.1: Конфигурирование приложения сводится к указанию путей в файловой системе.
    - *Путей к чему?* **(Каталог с исходными файлами, каталог с конечными файлами.)**
  - ПТ-2.2: Целевой кодировкой является UTF8.
    - *Предполагается ли указание иной целевой кодировки, или UTF8 используется в качестве целевой всегда?* **(Только UTF8, других не надо.)**
- ПТ-3: Просмотр журнала работы приложения.
  - ПТ-3.1: В процессе работы приложение должно выводить журнал своей работы в консоль и лог-файл.
    - *Каков формат журнала?* **(Дата-время, что и с чем делали, что получилось. Гляньте в логе апача, там нормально написано.)**
    - *Различаются ли форматы журнала для консоли и лог-файла?* **(Нет.)**
    - *Как определяется имя лог-файла?* **(Третий параметр при запуске. Если не указан — пусть будет converter.log рядом с php-скриптом.)**
  - ПТ-3.2: При первом запуске приложения лог-файл создаётся, а при последующих — дописывается.
    - *Как приложение различает свой первый и последующие запуски?* **(Никак.)**

        - *Какова реакция приложения на отсутствие лог-файла в случае, если это не первый запуск?* **(Создаёт. Тут идея в том, чтобы оно не затирало старый лог — и всё.)**

Бизнес-правила

- БП-1: Источник и приёмник файлов
    - БП-1.1: Каталоги, являющиеся источником исходны~~м~~ *(опечатка, исходны**х**)* **(Да.)** и приёмником конечных файлов, не должны совпадать.
        - *Какова реакция приложения в случае совпадения этих каталогов?* **(Пишет хелп и поясняет, что не так.)**
    - БП-1.2: Каталог, являющийся приёмником конечных файлов, не может быть подкаталогом источника *(предлагаем заменить слово «источника» на фразу «каталога, являющегося источником исходных файлов»)*. **(Хорошо, пусть будет так.)**

Атрибуты качества

- АК-1: Производительность
    - АК-1.1: Приложение должно обеспечивать скорость обработки данных 5 МБ/сек.
        - *При каких технических характеристиках системы?* **(i7, 4GB RAM)**
- АК-2: Устойчивость к входным данным
    - АК-2.1: Приложение должно обрабатывать входные файлы размером до 50 МБ включительно.
        - *Какова реакция приложения на файлы, размер которых превышает 50 МБ?* **(Не трогает.)**
    - АК-2.2: Если входной файл не является текстовым, приложение должно произвести обработку.
        - *Обработку чего должно произвести приложение?* **(Этого файла. Не важно, что станет с файлом, лишь бы скрипт не умер.)**

Здесь есть несколько важных моментов, на которые стоит обратить внимание:

- Ответы заказчика могут быть менее структурированными и последовательными, чем наши вопросы. Это нормально. Он может позволить себе такое, мы — нет.
- Ответы заказчика могут содержать противоречия (в нашем примере сначала заказчик писал, что параметрами, передаваемыми из командной строки, являются только два имени каталога, а потом сказал, что там же указывается имя лог-файла). Это тоже нормально, т.к. заказчик мог что-то забыть или перепутать. Наша задача — свести эти противоречивые данные воедино (если это возможно) и задать уточняющие вопросы (если это необходимо).
- В случае если с нами общается технический специалист, в его ответах вполне могут проскакивать технические жаргонизмы (как «хелп» в нашем примере). Не надо переспрашивать его о том, что это такое, если жаргонизм имеет однозначное общепринятое значение, но при доработке текста наша задача — написать то же самое строгим техническим языком. Если жаргонизм всё же непонятен — тогда лучше спросить (так, «хелп» — это всего лишь краткая помощь, выводимая консольными приложениями как подсказка о том, как их использовать).

Уровень продуктных требований (см. главу «Уровни и типы требований»{39}). Применим т.н. «самостоятельное описание» (см. главу «Источники и пути выявления требований»{37}) и улучшим требования. Поскольку мы уже получили много специфической технической информации, можно параллельно писать полноценную спецификацию требований. Во многих случаях, когда для оформления требований используется простой текст, для удобства формируется единый документ, который интегрирует в себе как пользовательские требования, так и детальные спецификации. Теперь требования принимают следующий вид.

Системные характеристики

- СХ-1: Приложение является консольным.
- СХ-2: Приложение разрабатывается на языке программирования PHP (причина выбора языка PHP отражена в пункте О-1 раздела «Ограничения», особенности и важные настройки интерпретатора PHP отражены в пункте ДС-1 раздела «Детальные спецификации»).
- СХ-3: Приложение является кроссплатформенным с учётом пункта О-4 раздела «Ограничения».

Пользовательские требования

- Также см. диаграмму вариантов использования.
- ПТ-1: Запуск и остановка приложения.
  - ПТ-1.1: Запуск приложения производится из консоли командой «php converter.php SOURCE_DIR DESTINATION_DIR [LOG_FILE_NAME]» (описание параметров приведено в разделе ДС-2.1, реакция на ошибки при указании параметров приведена в разделах ДС-2.2, ДС-2.3, ДС-2.4).
  - ПТ-1.2: Остановка приложения производится выполнением команды Ctrl+C в окне консоли, из которого было запущено приложение.
- ПТ-2: Конфигурирование приложения.
  - ПТ-2.1: Конфигурирование приложения сводится к указанию параметров командной строки (см. ДС-2).
  - ПТ-2.2: Целевой кодировкой преобразования текстов является кодировка UTF8 (также см. О-5).
- ПТ-3: Просмотр журнала работы приложения.
  - ПТ-3.1: В процессе работы приложение должно выводить журнал своей работы в консоль и лог-файл (см. ДС-4), имя которого определяется правилами, указанными в ДС-2.1.
  - ПТ-3.2: Формат журнала работы и лог файла указан в ДС-4.1, а реакция приложения на наличие или отсутствие лог-файла указана в ДС-4.2 и ДС-4.3 соответственно.

Бизнес-правила

- БП-1: Источник и приёмник файлов
  - БП-1.1: Каталоги, являющиеся источником исходных и приёмником конечных файлов, не должны совпадать (см. также ДС-2.1 и ДС-3.2).
  - БП-1.2: Каталог, являющийся приёмником конечных файлов, не может находиться внутри каталога, являющегося источником исходных файлов или его подкаталогов (см. также ДС-2.1 и ДС-3.2).

Атрибуты качества

- АК-1: Производительность
  - АК-1.1: Приложение должно обеспечивать скорость обработки данных не менее 5 МБ/сек на аппаратном обеспечении, эквивалентном следующему: процессор i7, 4 ГБ оперативной памяти, средняя скорость чтения/записи на диск 30 МБ/сек. Также см. О-6.
- АК-2: Устойчивость к входным данным
  - АК-2.1: Требования относительно форматов обрабатываемых файлов изложены в ДС-5.1.
  - АК-2.2: Требования относительно размеров обрабатываемых файлов изложены в ДС-5.2.
  - АК-2.3: Поведение приложения в ситуации обработки файлов с нарушениями формата определено в ДС-5.3.

Ограничения

- О-1: Приложение разрабатывается на языке программирования PHP, использование которого обусловлено возможностью заказчика осуществлять поддержку приложения силами собственного IT-отдела.
- О-2: Ограничения относительно версии и настроек интерпретатора PHP отражены в пункте ДС-1 раздела «Детальные спецификации».
- О-3: Процедуры установки и настройки интерпретатора PHP выходят за рамки данного проекта и **не описываются** в документации.
- О-4: Кроссплатформенные возможности приложения сводятся к способности работать под ОС семейства Windows и Linux, поддерживающих работу интерпретатора PHP версии, указанной в ДС-1.1.
- О-5: Целевая кодировка UTF8 является жёстко заданной, и её изменение в процессе эксплуатации приложения не предусмотрено.
- О-6: Допускается невыполнение АК-1.1 в случае, если невозможность обеспечить заявленную производительность обусловлена объективными внешними причинами (например, техническими проблемами на сервере заказчика).

Созданные на основе таких пользовательских требований детальные спецификации имеют следующий вид.

Детальные спецификации

ДС-1: Интерпретатор PHP

ДС-1.1: Минимальная версия — 5.5.

ДС-1.2: Для работы приложения должно быть установлено и включено расширение mbstring.

ДС-2: Параметры командной строки

ДС-2.1: При запуске приложения оно получает из командной строки три параметра:

SOURCE_DIR — обязательный параметр, определяет путь к каталогу с файлами, которые необходимо обработать;

DESTINATION_DIR — обязательный параметр, определяет путь к каталогу, в который необходимо поместить обработанные файлы (этот каталог не может находиться внутри каталога SOURCE_DIR или в его подкаталогах (см. БП-1.1 и БП-1.2));

LOG_FILE_NAME — необязательный параметр, определяет полное имя лог-файла (по умолчанию лог-файл с именем «converter.log» размещается по тому же пути, по которому находится файл скрипта converter.php);

ДС-2.2: При указании недостаточного количества параметров командной строки приложение должно завершить работу, выдав сообщение об использовании (ДС-3.1).

ДС-2.3: При указании излишнего количества параметров командной строки приложение должно игнорировать все параметры командной строки, кроме указанных в пункте ДС-2.1.

ДС-2.4: При указании неверного значения любого из параметров командной строки приложение должно завершить работу, выдав сообщение об использовании (ДС-3.1), а также сообщив имя неверно указанного параметра, его значение и суть ошибки (см. ДС-3.2).

ДС-3: Сообщения

ДС-3.1: Сообщение об использовании: «USAGE converter.php SOURCE_DIR DESTINATION_DIR LOG_FILE_NAME».

ДС-3.2: Сообщения об ошибках:

Directory not exists or inaccessible.
Destination dir may not reside within source dir tree.
Wrong file name or inaccessible path.

ДС-4: Журнал работы

ДС-4.1: Формат журнала работы одинаков для отображения в консоли и записи в лог-файл: YYYY-MM-DD HH:II:SS имя_операции параметры_операции результат_операции.

ДС-4.2: В случае если лог-файл отсутствует, должен быть создан новый пустой лог-файл.

ДС-4.3: В случае если лог-файл уже существует, должно происходить добавление новых записей в его конец.

ДС-5: Форматы и размеры файлов

ДС-5.1: Приложение должно обрабатывать текстовые файлы на русском и английском языках в следующих исходных кодировках: WIN1251, CP866, KOI8R.

Обрабатываемые файлы могут быть представлены в следующих форматах, определяемых расширениями файлов:

Plain Text (TXT);
Hyper Text Markup Language Document (HTML);
Mark Down Document (MD).

ДС-5.2: Приложение должно обрабатывать файлы размером до 50 МБ (включительно), игнорируя любой файл, размер которого превышает 50 МБ.

ДС-5.3: Если файл с расширением из ДС-5.1 содержит внутри себя данные, не соответствующие формату файла, допускается повреждение таких данных.



Задание 2.2.d: заметили ли вы, что в исправленном варианте требований «потерялась» диаграмма вариантов использования (равно как и активная ссылка на неё)? (Просто тест на внимательность, не более.)

Итак, мы получили набор требований, с которым уже вполне можно работать. Он не идеален (и никогда вы не встретите идеальных требований), но он вполне пригоден для того, чтобы разработчики смогли реализовать приложение, а тестировщики — протестировать его.



Задание 2.2.e: протестируйте этот набор требований и найдите в нём хотя бы 3–5 ошибок и неточностей, задайте соответствующие вопросы заказчику.

2.2.8. Типичные ошибки при анализе и тестировании требований

Для лучшего понимания и запоминания материала рассмотрим типичные ошибки, совершаемые в процессе анализа и тестирования требований.

Изменение формата файла и документа. По какой-то непонятной причине очень многие начинающие тестировщики стремятся полностью уничтожить исходный документ, заменив текст таблицами (или наоборот), перенеся данные из Word в Excel и т.д. Это можно сделать только в одном случае: если вы предварительно договорились о подобных изменениях с автором документа. В противном случае вы полностью уничтожаете чью-то работу, делая дальнейшее развитие документа крайне затруднительным.

Самое худшее, что можно сделать с документом, — это сохранить его в итоге в некоем формате, предназначенном скорее для чтения, чем для редактирования (PDF, набор картинок и тому подобное).

Если требования изначально создаются в некоей системе управления требованиями, этот вопрос неактуален, но высокоуровневые требования большинство заказчиков привыкли видеть в обычном DOCX-документе, а Word предоставляет такие прекрасные возможности работы с документом, как отслеживание изменений (см. рисунок 2.2.i) и комментарии (см. рисунок 2.2.j).

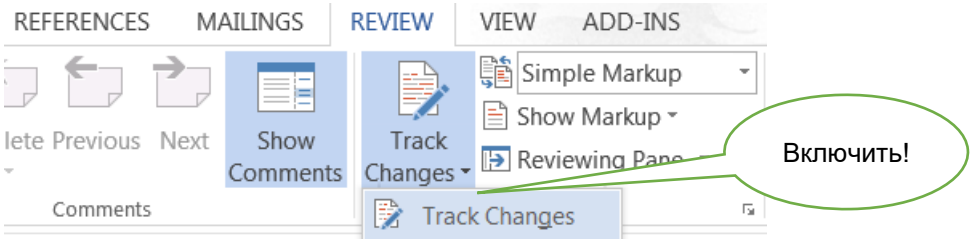


Рисунок 2.2.i — Активация отслеживания изменений в Word

В итоге получается результат, представленный на рисунке 2.2.j: исходный формат сохраняется (а автор к нему уже привык), все изменения хорошо видны и могут быть приняты или отклонены в пару кликов мыши, а типичные часто повторяющиеся вопросы вы можете помимо указания в комментариях вынести в отдельный список и поместить его в том же документе.

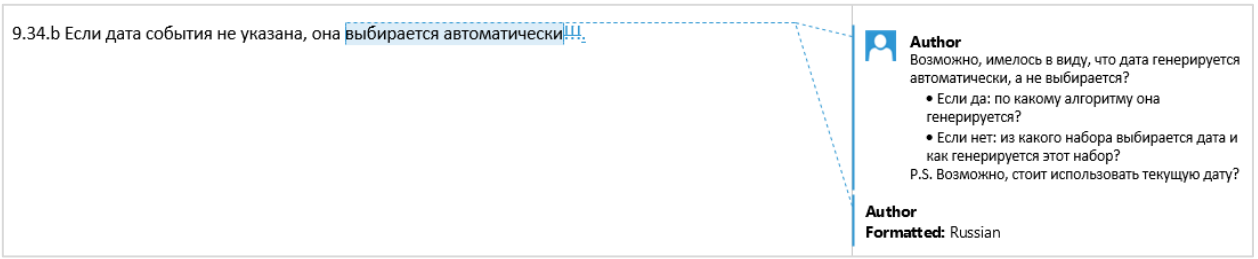


Рисунок 2.2.j — Правильно выглядящий документ с правками

И ещё два маленьких, но неприятных момента относительно таблиц:

- Выравнивание ВСЕГО текста в таблице по центру. Да, выравнивание по центру хорошо смотрится в заголовках и ячейках с парой-тройкой слов, но если так выровнен весь текст, читать его становится сложно.
- Отключение границ ячеек. Такая таблица намного хуже читается.

Отметка того факта, что с требованием всё в порядке. Если у вас не возникло вопросов и/или замечаний к требованию — не надо об этом писать. Любые пометки в документе подсознательно воспринимаются как признак проблемы, и такое «одобрение требований» только раздражает и затрудняет работу с документом — сложнее становится заметить пометки, относящиеся к проблемам.

Описание одной и той же проблемы в нескольких местах. Помните, что ваши пометки, комментарии, замечания и вопросы тоже должны обладать свойствами хороших требований (настолько, насколько эти свойства к ним применимы). Если вы много раз в разных местах пишете одно и то же об одном и том же, вы нарушаете как минимум свойство модифицируемости. Постарайтесь в таком случае вынести ваш текст в конец документа, укажите в нём же (в начале) перечень пунктов требований, к которым он относится, а в самих требованиях в комментариях просто ссылайтесь на этот текст.

Написание вопросов и комментариев без указания места требования, к которым они относятся. Если ваше инструментальное средство позволяет указать часть требования, к которому вы пишете вопрос или комментарий, сделайте это (например, Word позволяет выделить для комментирования любую часть текста — хоть один символ). Если это невозможно, цитируйте соответствующую часть текста. В противном случае вы порождаете неоднозначность или вовсе делаете вашу пометку бессмысленной, т.к. становится невозможно понять, о чём вообще идёт речь.

Задавание плохо сформулированных вопросов. Эта ошибка была подробно рассмотрена выше (см. раздел «Техники тестирования требований»{51} и таблицу 2.2.a{52}). Однако добавим, что есть ещё три вида плохих вопросов:

- Первый вид возникает из-за того, что автор вопроса не знает общепринятой терминологии или типичного поведения стандартных элементов интерфейса (например, «что такое чек-бокс?», «как в списке можно выбрать несколько пунктов?», «как подсказка может всплывать?»).
- Второй вид плохих вопросов похож на первый из-за формулировок: вместо того, чтобы написать «что вы имеете в виду под {чем-то}?», автор вопроса пишет «что такое {что-то}?» То есть вместо вполне логичного уточнения получается ситуация, очень похожая на рассмотренную в предыдущем пункте.
- Третий вид сложно привязать к причине возникновения, но его суть в том, что к некорректному и/или невыполнимому требованию задаётся вопрос наподобие «что будет, если мы это сделаем?». Ничего не будет, т.к. мы это точно не сделаем. И вопрос должен быть совершенно иным (каким именно — зависит от конкретной ситуации, но точно не таким).

И ещё раз напомним о точности формулировок: иногда одно-два слова могут на корню уничтожить отличную идею, превратив хороший вопрос в плохой. Сравните: «Что такое формат даты по умолчанию?» и «Каков формат даты по умолчанию?». Первый вариант просто показывает некомпетентность автора вопроса, тогда как второй — позволяет получить полезную информацию.

К этой же проблеме относится непонимание контекста. Часто можно увидеть вопросы в стиле «о каком приложении идёт речь?», «что такое система?» и им подобные. Чаще всего автор таких вопросов просто вырвал требование из контекста, по которому было совершенно ясно, о чём идёт речь.

Написание очень длинных комментариев и/или вопросов. История знает случаи, когда одна страница исходных требований превращалась в 20–30 страниц текста анализа и вопросов. Это плохой подход. Все те же мысли можно выразить значительно более кратко, чем сэкономить как своё время, так и время автора исходного документа. Тем более стоит учитывать, что на начальных стадиях работы с требованиями они весьма нестабильны, и может получиться так, что ваши 5–10 страниц комментариев относятся к требованию, которое просто удалят или изменят до неузнаваемости.

Критика текста или даже его автора. Помните, что ваша задача — сделать требования лучше, а не показать их недостатки (или недостатки автора). Потому комментарии вида «плохое требование», «неужели вы не понимаете, как глупо это звучит», «надо переформулировать» неуместны и недопустимы.

Категоричные заявления без обоснования. Как продолжение ошибки «критика текста или даже его автора» можно отметить и просто категоричные заявления наподобие «это невозможно», «мы не будем этого делать», «это не нужно». Даже если вы понимаете, что требование бессмысленно или невыполнимо, эту мысль стоит сформулировать в корректной форме и дополнить вопросами, позволяющими автору документа самому принять окончательное решение. Например, «это не нужно» можно переформулировать так: «Мы сомневаемся в том, что данная функция будет востребована пользователями. Какова важность этого требования? Уверены ли вы в его необходимости?»

Указание проблемы с требованиями без пояснения её сути. Помните, что автор исходного документа может не быть специалистом по тестированию или бизнес-анализу. Потому просто пометка в стиле «неполнота», «двусмысленность» и т.д. могут ничего ему не сказать. Поясняйте свою мысль.

Сюда же можно отнести небольшую, но досадную недоработку, относящуюся к противоречивости: если вы обнаружили некие противоречия, сделайте соответствующие пометки во всех противоречащих друг другу местах, а не только в одном из них. Например, вы обнаружили, что требование 20 противоречит требованию 30. Тогда в требовании 20 отметьте, что оно противоречит требованию 30, и наоборот. И поясните суть противоречия.

Плохое оформление вопросов и комментариев. Старайтесь сделать ваши вопросы и комментарии максимально простыми для восприятия. Помните не только о краткости формулировок, но и об оформлении текста (см., например, как на рисунке 2.2.j вопросы структурированы в виде списка — такая структура воспринимается намного лучше, чем сплошной текст). Перечитайте свой текст, исправьте опечатки, грамматические и пунктуационные ошибки и т.д.

Описание проблемы не в том месте, к которому она относится. Классическим примером может быть неточность в сноске, приложении или рисунке, которая почему-то описана не там, где она находится, а в тексте, ссылающемся на соответствующий элемент. Исключением может считаться противоречивость, при которой описать проблему нужно в обоих местах.

Ошибочное восприятие требования как «требования к пользователю». Ранее (см. «Корректность» в «Свойства качественных требований») мы говорили, что требования в стиле «пользователь должен быть в состоянии отправить сообщение» являются некорректными. И это так. Но бывают ситуации, когда проблема намного менее опасна и состоит только в формулировке. Например, фразы в стиле «пользователь может нажать на любую из кнопок», «пользователю должно быть видно главное меню» на самом деле означают «все отображаемые кнопки должны

быть доступны для нажатия» и «главное меню должно отображаться». Да, эту недоработку тоже стоит исправить, но не следует отмечать её как критическую проблему.

Скрытое редактирование требований. Эту ошибку можно смело отнести к разряду крайне опасных. Её суть состоит в том, что тестировщик произвольно вносит правки в требования, никак не отмечая этот факт. Соответственно, автор документа, скорее всего, не заметит такой правки, а потом будет очень удивлён, когда в продукте что-то будет реализовано совсем не так, как когда-то было описано в требованиях. Потому простая рекомендация: если вы что-то правите, обязательно отмечайте это (средствами вашего инструмента или просто явно в тексте). И ещё лучше отмечать правку как предложение по изменению, а не как свершившийся факт, т.к. автор исходного документа может иметь совершенно иной взгляд на ситуацию.

Анализ, не соответствующий уровню требований. При тестировании требований следует постоянно помнить, к какому уровню они относятся, т.к. в противном случае появляются следующие типичные ошибки:

- Добавление в бизнес-требования мелких технических подробностей.
- Дублирование на уровне пользовательских требований части бизнес-требований (если вы хотите увеличить прослеживаемость набора требований, имеет смысл просто использовать ссылки).
- Недостаточная детализация требований уровня продукта (общие фразы, допустимые, например, на уровне бизнес-требований, здесь уже должны быть предельно детализированы, структурированы и дополнены подробной технической информацией).